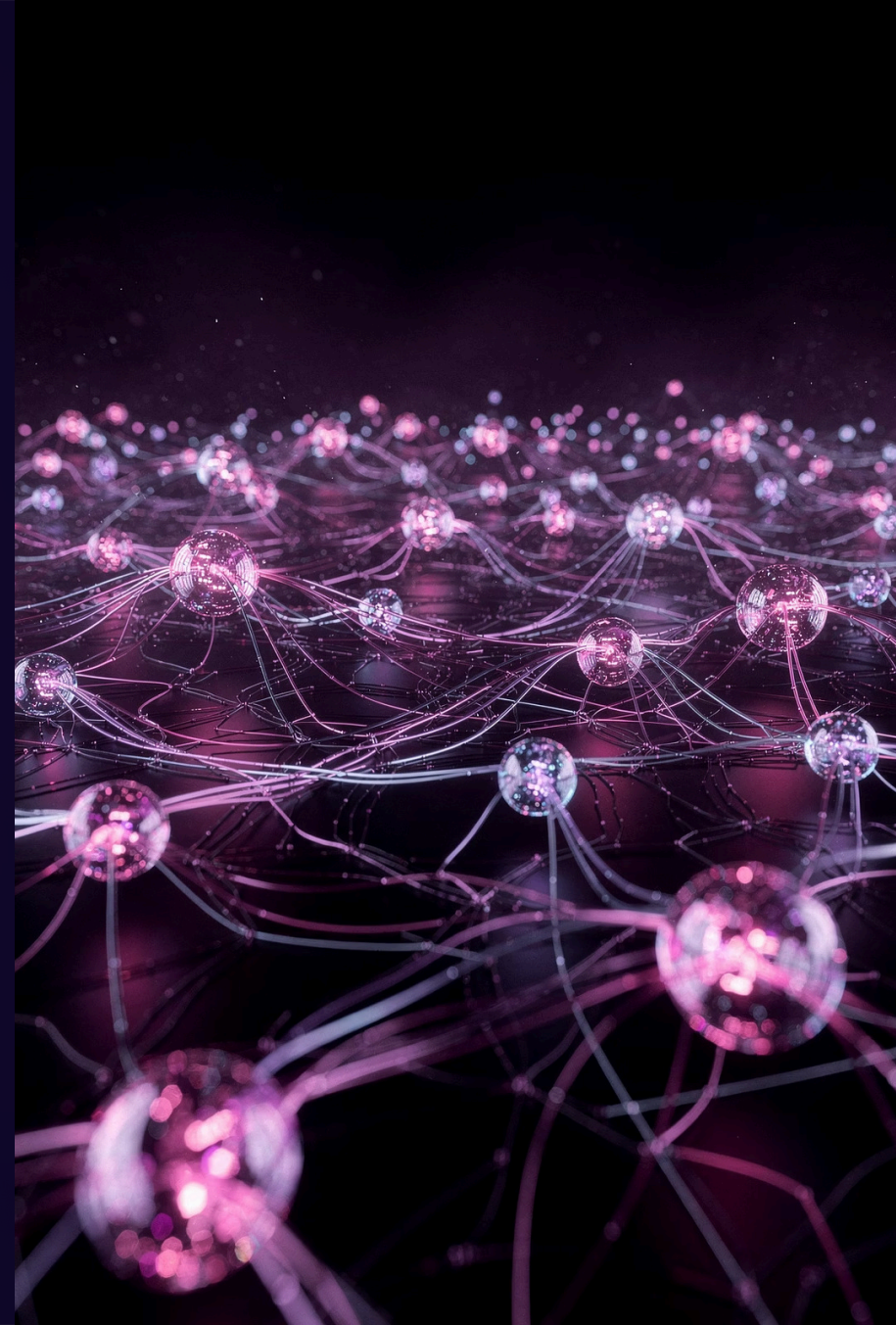


Agent Collaboration Patterns

How to organise multiple AI agents so they share work reliably — with clear roles, clean handoffs, and systems that are easy to reason about, debug, and scale.

[MULTI-AGENT SYSTEMS](#)[ARCHITECTURE GUIDE](#)

What Is an Agent Collaboration Pattern?

A repeatable way of organising agents

An agent collaboration pattern is a **repeatable way of organising agents** so they can share work reliably — for example, one planner plus several executors, or a router plus specialists.

Instead of inventing a new structure every time, you reuse patterns that define **who decides, who executes, and how information flows** between agents.

- ① Patterns give you **clear roles and responsibilities**, which makes systems easier to reason about and debug — and simpler to reuse across different problem domains.

Decide Once

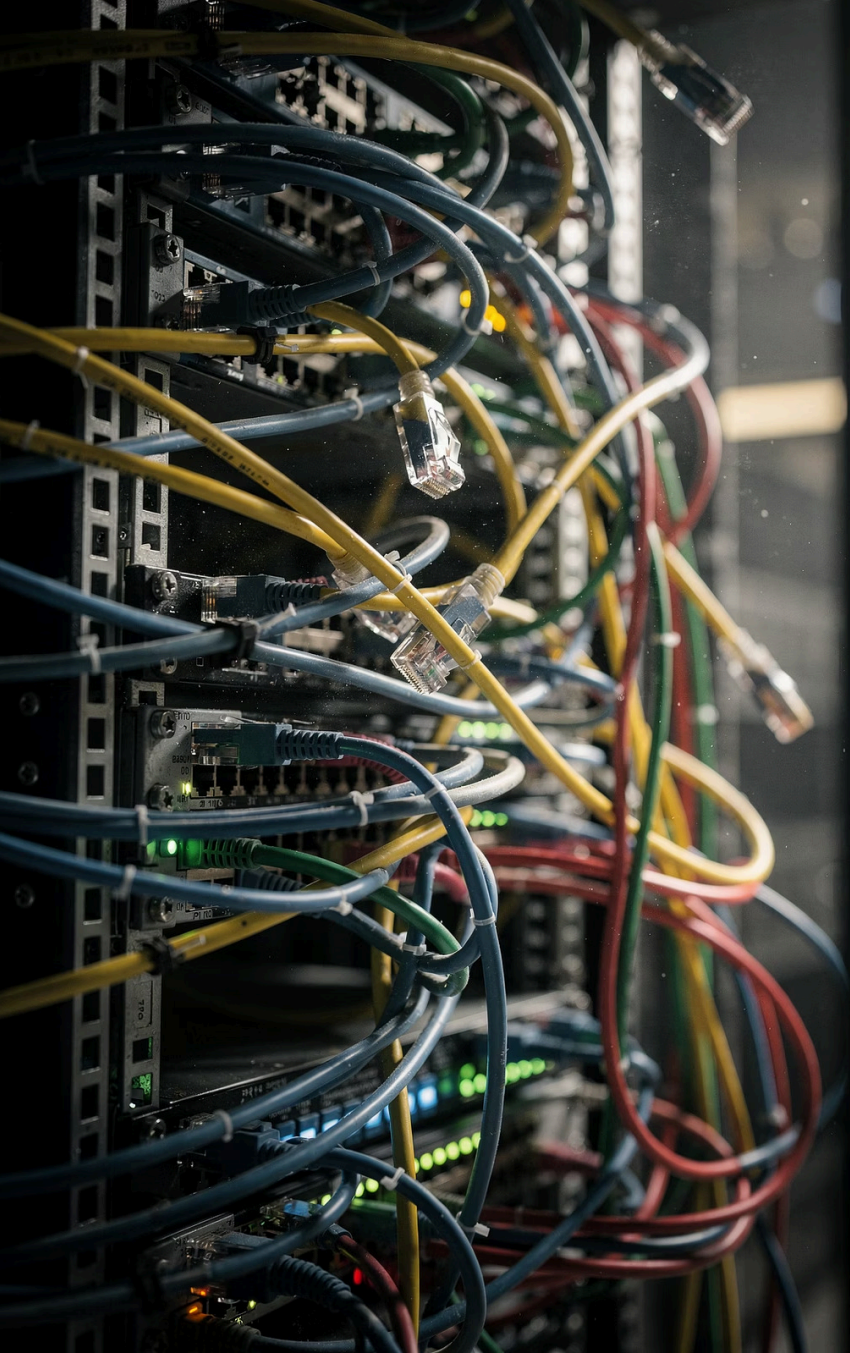
Define who plans, who routes, and who executes — up front.

Reuse Everywhere

Apply the same structural pattern across different projects and domains.

Debug Faster

Clear roles mean you always know where to look when something breaks.



Why Move from a Single Agent to Collaborating Agents?

Problems with a Single Agent

The prompt and logic become **large and tangled**, mixing support, sales, troubleshooting, and more in one place.

It becomes harder to attach the right tools and data sources to the right kind of task — and debugging specific behaviours is more painful.

How Collaborating Agents Help

You can split responsibilities: one agent **plans or routes**, while specialists handle specific task types like support or sales.

Each agent's scope becomes **narrower and more testable**, while the system as a whole can cover more complex scenarios.

The Planner—Executor Architecture

A foundational multi-agent pattern where reasoning and execution are cleanly separated into distinct roles.



The Planner

Interprets the user's request, breaks it into **steps or sub-tasks**, and decides which agent or tool should handle each step. Focuses on **reasoning and sequencing** — not on calling every tool or API directly.



The Executor

Specialist agents (or tightly scoped flows) that handle **specific types of tasks** — such as support, sales, or data lookups. They follow the Planner's guidance and return results that the Planner combines into a final answer.



The Combination

The Planner collects results from all Executors and **synthesises a coherent final response** — the user sees one unified answer, not fragmented outputs from multiple agents.

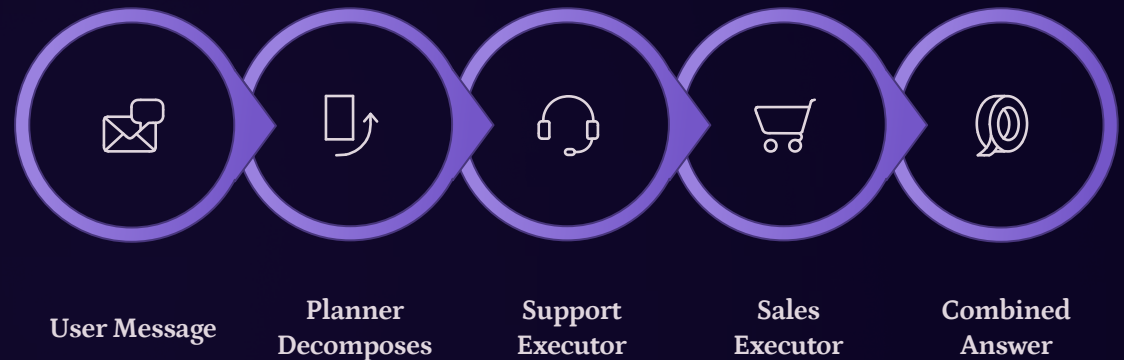
Planner—Executor Flow in Practice

A Customer Support Scenario

The Planner reads a user message containing both "**Where is my order?**" and "**Should I upgrade my phone?**" — two very different intents in one message.

It delegates the order question to a **Support Executor** and the recommendation question to a **Sales Executor**, then combines their answers into one reply.

- ✓ The Planner stays focused on **task decomposition and delegation**, while each Executor stays focused on doing one job well.



This leads to clearer logs, easier testing, and prompts that are smaller and more targeted than a single monolithic agent.

Task Delegation in a Multi-Agent System

What exactly is being delegated?

A Planner or Orchestrator delegates a **well-defined task** — such as "check this order's status" or "recommend two laptops within this budget" — to a specific agent.

The delegate receives enough context (order ID, user preferences) to do its job **without re-interpreting the entire conversation**.

What makes delegation robust?

Clear Inputs & Outputs

Agents can hand work off and pick it up again reliably when task boundaries are explicit.

Tracked Assignment

The orchestrating agent keeps track of which tasks have been assigned and how results should be combined.



How Handoff Works Within One Conversation

When a user's request shifts topics mid-conversation, the Orchestrator manages a seamless transition between specialist agents.

1

Detect the Shift

The Orchestrator recognises the conversation has moved — for example, from **order tracking** to **product recommendation**.

2

Transfer the Role

Active control passes from the **Support Executor** to the **Sales Executor**, with full context preserved.

3

Preserve Context

User profile, recent questions, and relevant IDs **travel with the handoff** so the next agent continues smoothly.

4

One Coherent Experience

The user experiences **one unified assistant**, even though different agents are taking turns behind the scenes.

- ❏ Shared context — user profile, recent questions, and relevant IDs — must travel with every handoff so the next agent can continue without asking the user to repeat themselves.

When Should the Orchestrator Keep Control vs. Hand Off?

Answer Directly

For very **simple or generic questions**, the Orchestrator can respond itself without bringing in a specialist — keeping the path short and efficient.

Hand Off to Specialist

For anything that clearly belongs to a domain — specific order issues or detailed recommendations — it should **delegate to the relevant specialist**.

Why this balance matters

Letting the Orchestrator handle trivial cases **avoids unnecessary complexity** and keeps latency low for simple interactions.

Consistent handoffs for domain-specific tasks keep each specialist's contract **clear and easy to test and tune** — you always know which agent owns which type of work.

- ① The goal is the **shortest reliable path** to a correct answer — not always the most elaborate routing.



Support and Sales Agents Collaborating Under an Orchestrator

Structuring the collaboration

The Orchestrator interprets an incoming message and decides whether it is **support-centric**, **sales-centric**, or mixed — then delegates accordingly within the same overall conversation.

Support Agent

Handles order status, returns, and problems — with tools and prompts tuned to resolution workflows.

Sales Agent

Handles comparisons and recommendations — with tools and prompts tuned to product knowledge and persuasion.

What this buys you

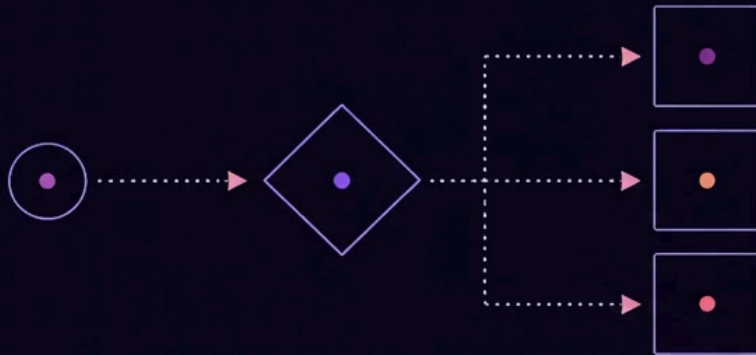
Each agent can have **prompts, tools, and evaluation criteria** tuned to its domain — no compromises needed.

Changes to support logic or sales strategy can be made in **one agent without destabilising the other**, making the system far easier to maintain and evolve over time.

Other Popular Multi-Agent Collaboration Patterns

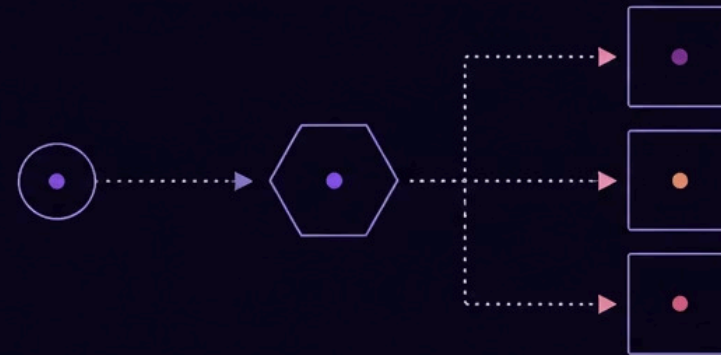
Planner-Executor

planner breaks tasks, executors specialise



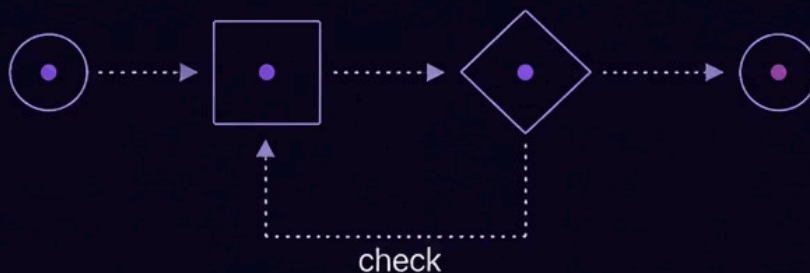
Specialist Router

lightweight routing agent classifies request and chooses specialist, no heavy reasoning



Verification Loop

one agent produces answer, second checker agent validates before user sees it



Simple Handoff

orchestrator passes control between agents as conversation topic shifts



When to use a Specialist Router

Choose this pattern when the main need is **correct initial routing** rather than complex planning — a lightweight classifier that picks the right specialist without heavy reasoning overhead.

When to use a Verification Loop

Use this when the **cost of mistakes is high** and you want a "second set of eyes" — a checker agent that critiques or validates answers before the user sees them, reducing errors and hallucinations.

Observability and Traceability for Multi-Agent Systems

What you need to see when things go wrong

You need a trace that shows **which agent handled each step**, what input it received, what tools it called, and what output it produced.

You also need to see the Planner or Orchestrator's decisions — so you can understand **why a particular agent was chosen** or why a handoff occurred at a given moment.

⚠ Without clear traces, debugging a multi-agent system becomes guesswork — you can't tell whether a failure was in routing, delegation, or execution.

How patterns make tracing easier

Clear patterns make it easier to know **what you should see in a trace** — for example:

01

Planner receives user message

Decomposes into sub-tasks and selects agents.

02

Support Agent executes

Handles order-related sub-task, returns result.

03

Sales Agent executes

Handles recommendation sub-task, returns result.

04

Planner combines and responds

If the trace deviates, that's your signal to refine routing or handoff logic.

What to Explore Next — Beyond a Single Planner



Multiple Orchestrators

How might multiple planners or orchestrators coordinate when a system grows beyond "support and sales" into many specialised domains?



Surfacing Agent Identity in UI

What would it look like for user interfaces to surface which agent handled which part of an answer, without overwhelming the user?



Per-Agent Evaluations & Guardrails

In what ways could evaluations and guardrails be tailored per agent, so that Support and Sales follow different quality and safety standards?



Long-Running & Async Tasks

How might long-running or asynchronous tasks fit into these collaboration patterns, especially when some steps take much longer than others?

